

Dispensa · Modulo 9

1. Dati reali: persistenti, grandi e strutturati

I dati volatili e il limite di `input()`

Fino ad ora i programmi che abbiamo scritto lavoravano su dati **volatili**: valori inseriti dall'utente con `input()`, oppure scritti direttamente nel codice. Questa volatilità deriva dall'architettura del calcolatore.

Nell'architettura di von Neumann — quella su cui si basa qualsiasi computer moderno — le operazioni di un programma Python avvengono nella **RAM** (Random Access Memory), la memoria di lavoro del processore. La RAM è veloce ma ha un difetto fondamentale: è **volatile**. Quando il programma termina, o quando il computer si spegne, tutto quello che c'era in RAM sparisce.

Questo non è un problema quando elaboriamo due numeri digitati da tastiera. Diventa un problema non appena vogliamo:

- lavorare su dati che esistevano prima dell'esecuzione del programma (per esempio, i voti di tutti gli studenti di un corso);
- salvare i risultati in modo da poterli rileggere in futuro.

La memoria persistente: disco e file

I **file** risolvono il problema della volatilità. Un file è una sequenza di dati salvata su una **memoria di massa** — hard disk, SSD, chiavetta USB, spazio cloud — che sopravvive alla chiusura del programma e allo spegnimento del computer. A differenza della RAM, la memoria di massa è lenta ma permanente.

Il sistema operativo identifica ogni file tramite un **percorso** che ne descrive la posizione nella struttura ad albero del filesystem. Quando Python vuole leggere o scrivere un file, lo comunica al sistema operativo attraverso questo percorso.

Tre caratteristiche dei dati reali

I dati che incontreremo nella pratica hanno tre proprietà che cambiano il modo in cui dobbiamo progettare i programmi.

Sono persistenti. Esistono indipendentemente dall'esecuzione del nostro codice. Il nostro script li legge, li elabora, produce un risultato — ma i dati originali rimangono intatti sul disco.

Sono potenzialmente grandi. Un file di testo può contenere milioni di righe. Non possiamo sempre caricarlo tutto in una lista e poi ragionare su di esso: la RAM ha una capacità limitata. Dobbiamo scegliere consapevolmente come leggere i dati — tutto in una volta, o riga per riga.

Sono strutturati. I dati raramente arrivano come numeri isolati. Arrivano come tabelle (CSV), strutture annidate (JSON), testi marcati (HTML). La strut-

tura del file determina anche quale struttura dati Python conviene usare per rappresentarlo in memoria.

Recap: leggere e scrivere file in Python

Il modulo 8 ha introdotto gli strumenti di base per lavorare con i file. Li richiamiamo qui in sintesi.

Per aprire un file si usa `open()`, che restituisce un **file handler** — un oggetto che rappresenta il collegamento al file su disco. Il costrutto `with` garantisce che il file venga chiuso automaticamente al termine del blocco indentato:

```
with open('dati/voti.txt', 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        # elabora i campi
```

I tre parametri essenziali di `open()`:

Parametro	Significato
percorso	Stringa con il percorso assoluto o relativo del file
modalità	'r' per leggere, 'w' per scrivere
encoding	Codifica del testo, quasi sempre 'utf-8'

Due operazioni frequenti sulle righe lette:

- `strip()` — rimuove spazi e il carattere di a capo (`\n`) che chiude ogni riga;
- `split(',')` — divide la riga in una lista di campi usando la virgola come separatore.

Composte insieme restituiscono una lista pulita:

```
riga = "Sandra,Rossi,informatica,30\n"
riga.strip().split(',')
# → ['Sandra', 'Rossi', 'informatica', '30']
```

Per scrivere su file si apre in modalità 'w' e si usa `print()` con il parametro `file=`:

```
with open('output.txt', 'w', encoding='utf-8') as f:
    print("risultato:", 42, file=f)
```

Attenzione: la modalità 'w' **sovrascrive** il file se esiste già.

In alternativa, l'output del programma si può indirizzare su file direttamente dalla shell con `>`, senza modificare il codice:

```
python3 script.py > output.txt
```

Nel primo caso è il programma a decidere dove scrivere; nel secondo è la shell a intercettare lo standard output.

Il nome del file di input o di output può essere passato come argomento da riga di comando tramite `sys.argv`, evitando di modificare il codice ogni volta che cambia il file da elaborare:

```
import sys
nome_file = sys.argv[1]
```

Il buffering: perché il file va chiuso Scrivere su disco è un'operazione lenta — molto più lenta di scrivere in RAM. Se Python scaricasse ogni singola `print()` sul disco immediatamente, il programma rallenterebbe enormemente.

Per evitarlo, Python adotta la stessa strategia che usiamo noi quando svuotiamo la lavastoviglie: se la credenza dei piatti è in sala, non portiamo ogni piatto a posto uno alla volta, facendo avanti e indietro per ogni singolo piatto. Raccogliamo un gruppo di piatti, li portiamo tutti insieme, poi torniamo per il gruppo successivo. Il costo del tragitto si ammortizza su più pezzi.

Python fa lo stesso: accumula le operazioni di scrittura in una zona temporanea di memoria chiamata **buffer**, e le scarica su disco tutte insieme quando il buffer è pieno o quando il file viene chiuso. Il costrutto `with` garantisce la chiusura automatica all'uscita dal blocco, assicurando che il contenuto del buffer venga effettivamente scritto. Dimenticare di chiudere un file aperto in scrittura può causare perdita di dati.

2. Ispezionare i file dal terminale

Quando riceviamo un file di dati, prima di scrivere uno script è utile dargli un'occhiata: capire come è strutturato, qual è il separatore tra i campi, se ci sono righe di intestazione. Aprire un file da VS Code o da Excel va bene per file piccoli, ma non è sempre possibile: i file di grandi dimensioni possono non entrare in RAM, e quando si lavora su un server remoto spesso si ha solo un terminale.

Bash mette a disposizione alcune utilità leggere progettate esattamente per questo.

ls -lh

Il comando `ls` elenca il contenuto di una cartella. Con le opzioni `-lh` mostra informazioni aggiuntive in un formato leggibile:

```
$ ls -lh dati/
-rw-r--r-- 1 ludovica staff 4.0K 3 giu 10:15 numeri.txt
-rw-r--r-- 1 ludovica staff 66K 27 mag 11:42 testo.txt
-rw-r--r-- 1 ludovica staff 28K 27 mag 11:42 voti_materie.txt
```

Le colonne rilevanti: la dimensione del file (qui 4.0K, 66K, 28K) e la data dell'ultima modifica. Sapere quanto è grande un file aiuta a capire se ha senso leggerlo tutto in memoria o conviene farlo riga per riga.

cat

cat stampa tutto il contenuto di un file sul terminale:

```
$ cat dati/numeri.txt
3
17
42
8
```

È utile per file corti. Per file lunghi l'output scorre via troppo velocemente.

less

less apre il file in un visualizzatore interattivo: mostra una schermata alla volta e carica solo la parte visibile, quindi funziona bene anche con file di milioni di righe.

```
$ less dati/voti_materie.txt
```

I tasti principali dentro less:

Tasto	Azione
freccia su/giù	scorrere riga per riga
barra spazio	pagina successiva
q	uscire

Nota: less non è disponibile nativamente su Windows PowerShell. Se si usa Windows, conviene aprire il terminale Git Bash incluso nell'installazione di Git.

Altre utility: head, tail, wc, grep, cut, sort

Oltre a cat e less, bash offre una serie di strumenti specializzati per esplorare e trasformare file di testo. Ognuno fa una cosa sola, ma possono essere combinati.

head e tail head mostra le prime righe di un file (10 per default); tail le ultime. Utili per controllare rapidamente la struttura di un file senza aprirlo tutto.

```
head dati/voti.txt      # prime 10 righe
tail -5 dati/voti.txt   # ultime 5 righe
```

wc **wc** (word count) conta righe, parole e caratteri di un file. L'opzione **-l** restringe l'output al solo conteggio delle righe.

```
wc dati/voti.txt           # righe parole caratteri nome_file
wc -l dati/voti.txt        # solo il numero di righe
```

grep **grep** filtra le righe di un file che corrispondono a un pattern. Restituisce solo le righe che contengono quella stringa.

```
grep ",30" dati/voti.txt           # righe con voto 30
grep ",fisica," dati/voti_materie.txt # righe della materia fisica
```

L'opzione **-v** inverte il filtro — restituisce le righe che **non** contengono il pattern:

```
grep -v '^-' dati/numeri.txt      # righe che non iniziano con '-' (numeri positivi)
```

L'opzione **-E** attiva le espressioni regolari estese, permettendo per esempio l'alternanza con **|**:

```
grep -E "lista|dizionario" dati/testo.txt      # righe che contengono "lista" o "dizionario"
```

cut **cut** estrae una o più colonne da un file strutturato. L'opzione **-d** specifica il delimitatore, **-f** specifica il numero della colonna da estrarre.

```
cut -d',' -f2 dati/voti.txt           # seconda colonna (cognomi)
cut -d',' -f3 dati/voti_materie.txt    # terza colonna (materie)
```

sort **sort** ordina le righe di un file alfabeticamente. L'opzione **-n** ordina numericamente anziché alfabeticamente (senza **-n**, "9" verrebbe dopo "10" perché '9' > '1').

```
sort dati/testo.txt                 # ordine alfabetico
sort -n dati/numeri.txt             # ordine numerico crescente
```

Per ordinare su una colonna specifica di un CSV si usano **-t** (delimitatore) e **-k** (numero di colonna):

```
sort -t',' -k4 -n dati/voti_materie.txt      # ordina per quarta colonna (voto), numericamente
```

La pipe |

Ogni utility fa una cosa sola: **wc** conta, **sort** ordina, **grep** filtra. La potenza vera emerge quando si collegano più comandi in sequenza tramite la **pipe** **|**, che passa l'output di un comando come input del successivo, senza creare file intermedi.

```
comando1 | comando2 | comando3
```

È come una catena di montaggio: ogni passaggio riceve il risultato del precedente e lo trasforma.

Esempio: quante parole distinte contiene un testo?

```
cat testo.txt | tr ' ' '\n' | sort | uniq | wc -l
```

Passo per passo:

- `cat testo.txt` — stampa il file intero
- `tr ' ' '\n'` — sostituisce ogni spazio con un a capo: una parola per riga
- `sort` — ordina le righe alfabeticamente (necessario perché `uniq` elimina solo duplicati *consecutivi*)
- `uniq` — elimina le righe duplicate consecutive
- `wc -l` — conta le righe rimaste

Altri esempi utili con i file della repository:

```
# Quanti cognomi distinti ci sono in voti.txt?  
cut -d',' -f2 dati/voti.txt | sort | uniq | wc -l
```

```
# Quali materie compaiono in voti_materie.txt?  
cut -d',' -f3 dati/voti_materie.txt | sort | uniq
```

```
# Tutte le righe di informatica, ordinate per voto  
grep ",informatica," dati/voti_materie.txt | sort -t',' -k4 -n
```

Il risultato di una pipeline si può salvare su file con >:

```
cut -d',' -f3 dati/voti_materie.txt | sort | uniq > materie.txt
```

3. Dizionari

Perchè abbiamo bisogno dei dizionari?

Supponiamo di avere un file `voti_materie.txt` in cui ogni riga descrive il voto di uno studente in una materia:

```
Sandra,Rossi,matematica,28  
Matteo,Bianchi,matematica,22  
Sandra,Rossi,informatica,30  
Sandra,Rossi,linguistica,27  
Matteo,Bianchi,linguistica,25
```

Vogliamo calcolare la media dei voti per ogni studente. Il problema è che ogni studente appare su più righe: dobbiamo raccogliere tutti i suoi voti prima di poter fare la media.

Con una lista potremmo farlo, ma dovremmo scorrere il file più volte — una per ogni studente. Quello che vorremmo è una struttura che associa direttamente il nome dello studente alla lista dei suoi voti: cercando “Sandra Rossi” troviamo subito [28, 30, 27].

Questa struttura si chiama **dizionario**.

Cos'è un dizionario

Un dizionario è una collezione di coppie **chiave** → **valore**. Invece di usare una posizione numerica per accedere agli elementi (come nelle liste), si usa una chiave di qualsiasi tipo immutabile.

```
# lista: accesso per posizione
voti = [28, 30, 27]
voti[0]    # 28
```

```
# dizionario: accesso per chiave
d = {"Sandra Rossi": [28, 30, 27], "Matteo Bianchi": [22, 25]}
d["Sandra Rossi"]    # [28, 30, 27]
```

Il dizionario è **non ordinato**: non esiste una “prima” o “ultima” coppia nel senso in cui esiste il primo elemento di una lista. Quello che conta è la chiave.

Creare un dizionario

Un dizionario vuoto si crea con le parentesi graffe:

```
d = {}
```

Un dizionario già popolato si scrive elencando le coppie chiave-valore separate da virgola:

```
frequenze = {"the": 156, "cat": 20, "is": 80}
capoluoghi = {"Lazio": "Roma", "Campania": "Napoli", "Sicilia": "Palermo"}
province = {"Molise": ["Campobasso", "Isernia"], "Basilicata": ["Potenza", "Matera"]}
```

Il valore può essere di qualsiasi tipo: numero, stringa, lista, un altro dizionario. La chiave deve essere di un tipo **immutabile** (stringa, intero, float — non una lista).

Perché le chiavi devono essere immutabili? Le funzioni di hash.

Quando scriviamo `d["gatto"]`, Python deve trovare il valore corrispondente. Le liste supportano già l'accesso diretto per indice numerico: `lista[3]` è immediato, non scorre niente. Il problema è che un indice numerico non porta alcun significato — per trovare la posizione associata a una chiave come `"gatto"` bisognerebbe scorrere tutta la lista. Python risolve il problema con una strategia diversa: calcola **direttamente** la posizione di memoria del valore a partire dalla chiave, senza cercare niente.

Lo strumento per farlo si chiama **funzione di hash**: una funzione matematica che prende in ingresso un dato (la chiave) e restituisce sempre lo stesso numero intero — chiamato *hash* — a partire da quel dato. Per esempio, applicando la funzione di hash alla stringa `"gatto"` si ottiene sempre il numero 8765432 (il numero preciso

dipende dall'implementazione, ma è sempre lo stesso). Python usa quel numero per decidere in quale cella di memoria mettere il valore 5.

Quando poi scriviamo `d["gatto"]` per recuperare il valore, Python ricalcola l'hash di "gatto", ottiene di nuovo 8765432, va direttamente a quella cella e trova il valore. Zero ricerca, costo costante.

Questo meccanismo funziona solo se la chiave **non cambia mai**: se la chiave potesse modificarsi dopo essere stata inserita, l'hash calcolato al momento dell'inserimento sarebbe diverso dall'hash calcolato al momento della ricerca, e il valore non verrebbe trovato. Per questo le chiavi devono essere di tipo immutabile — stringhe, interi, float — e non possono essere liste, che invece sono mutabili.

Lo stesso principio vale per i set: anche lì ogni elemento occupa una cella determinata dal suo hash, ed è per questo che gli elementi di un set devono anch'essi essere immutabili.

Operazioni fondamentali

Accedere a un valore

```
d = {"gatto": 5, "cane": 7}
d["gatto"]    # 5
```

Se la chiave non esiste, Python solleva un `KeyError`:

```
d["elefante"]    # KeyError: 'elefante'
```

Per verificare prima se una chiave esiste, si usa l'operatore `in`:

```
if "gatto" in d:
    print(d["gatto"])
```

Assegnare o aggiornare un valore La sintassi è identica a quella delle liste, con la chiave al posto dell'indice:

```
d["gatto"] = 34    # aggiorna se la chiave esiste già
d["topo"] = 1      # aggiunge una nuova coppia
```

Numero di elementi

```
len(d)    # numero di chiavi nel dizionario
```

Chiavi, valori e coppie

```
d.keys()    # tutte le chiavi
d.values()  # tutti i valori
d.items()   # tutte le coppie (chiave, valore)
```


Iterare su un dizionario

Il `for` su un dizionario restituisce le chiavi:

```
d = {"gatto": 5, "cane": 7}
```

```
for chiave in d:
    print(chiave, d[chiave])
```

Output:

```
gatto 5
cane 7
```

Per ottenere chiave e valore insieme in un unico passaggio si usa `d.items()`, che restituisce coppie che il `for` può spaccettare in due variabili:

```
for chiave, valore in d.items():
    print(chiave, valore)
```

L'output è identico. Qual è la differenza? `for chiave in d` assegna solo la chiave, e il valore si recupera poi con `d[chiave]`. `for chiave, valore in d.items()` assegna entrambi direttamente. È una questione di leggibilità.

Perché non il `while`? Il `while` richiede che il programmatore esprima una condizione di terminazione — per esempio “finché l'indice è minore della lunghezza”. Ma il dizionario non ha indici: è Python a organizzare internamente le chiavi. Non abbiamo un modo diretto per scrivere quella condizione. Il `for` gestisce questa complessità per noi.

Esempio completo: media voti per studente

Il file `dati/voti_materie.txt` ha questo formato (una riga per voto):

```
Sandra,Rossi,matematica,28
Matteo,Bianchi,matematica,22
Sandra,Rossi,informatica,30
Sandra,Rossi,linguistica,27
Matteo,Bianchi,linguistica,25
```

Vogliamo stampare la media dei voti per ogni studente. Costruiamo il programma per passi.

Passo 1 — leggere il file e controllare il parsing.

Prima verifichiamo di saper leggere correttamente ogni riga:

```
import sys
```

```
nome_file = sys.argv[1]
```

```

with open(nome_file, 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        print(campi)

```

Output atteso per le prime righe:

```

['Sandra', 'Rossi', 'matematica', '28']
['Matteo', 'Bianchi', 'matematica', '22']
...

```

Se i campi sono quelli giusti, possiamo andare avanti.

Passo 2 — estrarre nome e voto da ogni riga.

Il nome completo è `campi[0] + ' ' + campi[1]`; il voto è `campi[3]`, ma arriva come stringa e va convertito in intero:

```

with open(nome_file, 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        nome = campi[0] + ' ' + campi[1]
        voto = int(campi[3])
        print(nome, voto)

```

Output:

```

Sandra Rossi 28
Matteo Bianchi 22
Sandra Rossi 30
...

```

Passo 3 — raccogliere i voti in un dizionario.

Vogliamo che `voti["Sandra Rossi"]` sia la lista `[28, 30, 27]`. La prima volta che incontriamo uno studente, la chiave non esiste ancora: dobbiamo crearla con una lista vuota prima di aggiungere il voto.

```
voti = {}
```

```

with open(nome_file, 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        nome = campi[0] + ' ' + campi[1]
        voto = int(campi[3])

        if nome not in voti:
            voti[nome] = []           # prima occorrenza: crea la lista
        voti[nome].append(voto)      # aggiunge il voto alla lista

print(voti)

```

Output:

```
{'Sandra Rossi': [28, 30, 27], 'Matteo Bianchi': [22, 25]}
```

Questo schema — `if chiave not in d: d[chiaive] = []` seguito da `d[chiaive].append(...)` — è il pattern standard per costruire un dizionario che accumula valori in una lista.

Passo 4 — calcolare e stampare le medie.

Definiamo una funzione che calcola la media di una lista di numeri. Restituisce un valore, quindi deve usare `return`:

```
def calcola_media(lista):
    somma = 0
    for numero in lista:
        somma = somma + numero

    return somma / len(lista)
```

Con il dizionario costruito, iteriamo sulle coppie chiave-valore e chiamiamo la funzione per ogni studente:

```
for studente, lista_voti in voti.items():
    media = calcola_media(lista_voti)
    print(studente, '→', f"{media:.2f}")
```

Script completo:

```
import sys

def calcola_media(lista):
    somma = 0
    for numero in lista:
        somma = somma + numero

    return somma / len(lista)

nome_file = sys.argv[1]
voti = {}

with open(nome_file, 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        nome = campi[0] + ' ' + campi[1]
        voto = int(campi[3])

        if nome not in voti:
            voti[nome] = []
        voti[nome].append(voto)
```

```
for studente, lista_voti in voti.items():
    media = calcola_media(lista_voti)
    print(studente, '→', f"{media:.2f}")
```

Esecuzione:

```
$ python3 media_per_studente.py dati/voti_materie.txt
Sandra Rossi → 28.33
Matteo Bianchi → 23.5
```

3. Set

Cos'è un set

Un **set** è una collezione di elementi unici e non ordinati. È come un dizionario che ha solo le chiavi, senza valori. Gli elementi devono essere immutabili (stesse regole delle chiavi dei dizionari).

Le proprietà fondamentali:

- **nessun duplicato**: aggiungere un elemento già presente non cambia il set e non produce errori;
- **nessun ordine**: non esiste un “primo” o “ultimo” elemento;
- **ricerca efficiente**: controllare se un elemento è nel set è molto più veloce che controllare se è in una lista (stessa logica dell'hash).

Creare un set

Le parentesi graffe vuote {} creano un dizionario, non un set. Per un set vuoto si usa la parola chiave **set**:

```
s = set()    # set vuoto
```

Per un set già popolato si possono usare le parentesi graffe con valori (senza i due punti dei dizionari):

```
animali = {"gatto", "cane", "topo"}
```

set() accetta qualsiasi iterabile e ne conserva solo gli elementi unici:

```
set("linguistica")
# → {'l', 'i', 'n', 'g', 'u', 's', 't', 'c', 'a'}
# crea un set dai caratteri della stringa (ognuno presente una sola volta)

set(["elefante", "cammello", "pentola", "elefante"])
# → {'elefante', 'cammello', 'pentola'}
# crea un set da una lista, eliminando i duplicati
```

Operazioni fondamentali

Aggiungere un elemento

```
animali = {"gatto", "cane"}
animali.add("topo")      # aggiunge topo
animali.add("gatto")     # gatto c'era già: nessuna modifica, nessun errore
```

Rimuovere un elemento

```
animali.remove("cane")
```

Verificare l'appartenenza

```
"gatto" in animali      # True
"elefante" in animali   # False
```

Numero di elementi

```
len(animali)
```

Operazioni insiemistiche

I set supportano le operazioni dell'algebra degli insiemi:

Operazione	Sintassi	Significato
Unione	$A \mid B$	Elementi in A o in B (o in entrambi)
Intersezione	$A \& B$	Elementi presenti sia in A che in B
Differenza	$A - B$	Elementi in A ma non in B

```
A = {"gatto", "cane", "topo"}
B = {"cane", "elefante"}
```

```
A | B      # {"gatto", "cane", "topo", "elefante"}
A & B      # {"cane"}
A - B      # {"gatto", "topo"}
```

Quando usare un set

Il caso d'uso principale è **eliminare i duplicati** da una collezione, oppure rispondere rapidamente alla domanda “questo elemento c'è?”.

Esempio: trovare tutti gli studenti distinti in un file.

```
studenti = set()
```

```
with open('dati/voti_materie.txt', 'r', encoding='utf-8') as f:
    for riga in f:
```

```

        campi = riga.strip().split(',')
        nome = campi[0] + ' ' + campi[1]
        studenti.add(nome)

print("Studenti nel corso:", len(studenti))
for s in studenti:
    print(s)

```

Ogni volta che `add()` riceve un nome già presente, lo ignora silenziosamente. Alla fine `studenti` contiene ogni nome una sola volta.

Set vs lista: quando fa differenza

Per l'operazione `in`, un set è strutturalmente più efficiente di una lista. Nella lista l'interprete deve scorrere tutti gli elementi uno per uno finché non trova quello cercato — nel caso peggiore, fino alla fine. Nel set, la funzione di hash calcola direttamente la posizione: il costo è lo stesso sia che il set contenga 10 elementi sia che ne contenga un milione.

Per la maggior parte degli esercizi di questo corso la differenza non è percepibile. Diventa rilevante quando si lavora con insiemi molto grandi o quando si esegue l'operazione `in` milioni di volte.

4. Scegliere la struttura dati giusta

Struttura	Ordine	Duplicati	Accesso	Quando usarla
Lista	Sì	Sì	Per posizione (indice)	Sequenza ordinata; si accede per posizione
Dizionario	No	No (chiavi)	Per chiave	Associare informazioni a un nome; raggruppare dati
Set	No	No	Solo appartenenza	Raccogliere elementi unici; testare appartenenza velocemente

La scelta dipende da cosa si vuole fare con i dati:

- Se i dati devono restare in ordine e si accede per posizione → **lista**
- Se si vuole associare ogni elemento a un'informazione aggiuntiva → **dizionario**
- Se si vuole solo sapere quali elementi distinti esistono → **set**